==========

==========

CHAPTER 5 — PAGE TABLE LEVELS AND DESCRIPTOR FORMATS

==========

==========

A maximum-detail, RTOS/QNX-style technical breakdown of ARMv8 page table hierarchy, descriptor formats, block/page mappings, and walk behavior.

****

****

5.1 — Introduction to Page Tables

****

****

ARMv8 uses a **multi-level page table hierarchy** to translate virtual addresses into physical addresses. The number of levels depends on:

- granule size
- virtual address size

- TCR configuration

Page tables contain **descriptors**, which define:

- physical base address
- memory attributes
- permissions
- shareability
- access flags
- execute permissions
- next-level table pointers

Descriptors are the fundamental building blocks of the MMU.

****

****

5.2 — Page Table Hierarchy Overview

****

****

With **4 KB granule**, ARMv8 uses up to **4 levels**:

```
L0 → optional (for >48-bit VA)
L1 → mandatory
L2 → mandatory
L3 → mandatory
```

Diagram (4 KB granule)

```
VA[47:39] → L0 index (optional)
VA[38:30] → L1 index
```

```
VA[29:21] → L2 index
VA[20:12] → L3 index
VA[11:0]  → page offset
```

Teaching Note

L0 is only used for VA sizes above 48 bits.

****

****

5.3 — Page Table Entry Size

****

****

Each descriptor is **64 bits**.

Each page table contains:

- **512 entries** (for 4 KB granule)
- **8 bytes per entry**
- **4096 bytes total**

This aligns perfectly with the granule size.

****

****

5.4 — Descriptor Types

****

****

Descriptor type is determined by bits [1:0]:

Bits	Type	Meaning
00	Invalid	No mapping
01	Block	Final mapping (L1/L2 only)
11	Table	Pointer to next level (L0/L1/L2)
11	Page	Final mapping (L3 only)

Teaching Note

Blocks map large regions; pages map small ones.

****

****

5.5 — Block vs Page Mappings

****

****

Block Mapping

Used at:

- L1 (1 GB blocks)
- L2 (2 MB blocks)

Page Mapping

Used at:

- L3 (4 KB pages)

Implementation Note

Blocks reduce TLB pressure and improve performance.

****

****

5.6 — Descriptor Format (Table Descriptor)

****

****

A **table descriptor** points to the next level.

```
63:12 → Next-level table base address
11:2  → Reserved (must be zero)
1:0   → 11 (table)
```

Additional bits:

- NSTable → Non-secure table
- APTable → Access permissions for next level
- UXNTable → Execute never for next level
- PXNTable → Privileged execute never for next level

Teaching Note

Table descriptors do not define memory attributes.

****

****

5.7 — Descriptor Format (Block/Page Descriptor)

****

****

Block/page descriptors define:

- physical base address
- permissions
- attributes
- shareability
- execute permissions

Format (simplified)

```
63:12 → Physical base address
11:10 → AF (Access Flag)
9:8   → SH (Shareability)
7:5   → AP (Access Permissions)
4:2   → AttrIndx (MAIR index)
1:0   → Type (01 = block, 11 = page)
```

Implementation Note

AF must be set for valid mappings.

****

****

5.8 — Access Flag (AF)

****

****

AF indicates whether the page has been accessed.

- **AF = 1** → page is accessible
- **AF = 0** → access generates Access Flag Fault

Teaching Note

OS typically sets AF=1 for all pages.

****

****

5.9 — Access Permissions (AP)

****

****

AP controls read/write permissions.

AP	Meaning
00	EL0/EL1 read/write

01	EL1 read/write, EL0 no access
10	EL0/EL1 read-only
11	EL1 read-only, EL0 no access

Implementation Note

AP interacts with SCTLR_EL1.WXN and PAN.

****

****

5.10 — Execute Permissions (PXN/UXN)

****

****

PXN → Privileged Execute Never UXN → User Execute Never

Bit	Meaning
PXN	EL1 cannot execute
UXN	EL0 cannot execute

Teaching Note

Used to enforce W^X (Write-xor-Execute).

****

****

5.11 — Shareability (SH)

****

****

SH controls cache coherence.

SH	Meaning
00	Non-shareable
10	Outer Shareable
11	Inner Shareable

Implementation Note

Inner Shareable is required for SMP.

****

****

5.12 — Memory Attributes (AttrIndx)

****

****

AttrIndx selects MAIR entry.

Example:

```
AttrIndx = 2 → MAIR_EL1[2]
```

Teaching Note

Memory type affects caching, ordering, and device behavior.

****

****

5.13 — Physical Address Encoding

****

****

Physical base address is stored in bits [47:12].

Alignment requirements:

- L1 block → 1 GB aligned
- L2 block → 2 MB aligned
- L3 page → 4 KB aligned

Implementation Note

Misaligned PA causes Translation Fault.

****

****

5.14 — Next-Level Table Address Encoding

****

****

Table descriptors store next-level table base address in bits [47:12].

Alignment:

- must be 4 KB aligned
- must point to valid memory
- must not point to device memory

****

****

5.15 — Descriptor Validity Rules

****

****

A descriptor is valid if:

- type bits are correct
- AF is set
- PA is aligned

- AttrIndx references valid MAIR entry
- SH is legal for memory type
- AP is legal for privilege level

Teaching Note

Invalid descriptors cause Translation Fault.

****

****

5.16 — Page Table Walk Behavior

****

****

Walk steps:

1. Select TTBR
2. Read L1 descriptor
3. If table → read L2
4. If table → read L3
5. If page/block → final mapping
6. Apply permissions
7. Apply attributes
8. Produce IPA (Stage-1) or PA (Stage-2)

Implementation Note

Walk cache accelerates steps 2–4.

****

****

5.17 — Page Table Walk Faults

****

****

Faults include:

- Invalid descriptor
- Misaligned descriptor
- Access Flag Fault
- Permission Fault
- Translation Fault
- Alignment Fault

Teaching Note

Faults occur at the earliest possible stage.

****

****

5.18 — Example: Full Descriptor Walkthrough

****

****

Given VA:

`0x0000_7FFF_1234_5678`

Step 1 — TTBR selection

TTBR0_EL1.

Step 2 — L1 descriptor

Type = table → go to L2.

Step 3 — L2 descriptor

Type = block → final mapping.

Step 4 — Extract fields

- PA base
- AP
- SH
- AttrIndx
- PXN/UXN

Step 5 — Produce IPA

IPA = PA_base + offset.

****

****

5.19 — Example: Page Descriptor (L3)

****

****

```
Descriptor = 0x0000_1234_5600_0C0F
```

Breakdown:

- PA = 0x0000_1234_5600_0000
- AF = 1
- SH = Inner Shareable
- AP = EL0/EL1 read/write
- AttrIndx = 0 (Normal WB)
- UXN = 1 (EL0 cannot execute)
- PXN = 0 (EL1 can execute)

****

****

5.20 — Example: Block Descriptor (L2)

****

****

```
Descriptor = 0x0000_2000_0000_0C01
```

Breakdown:

- PA = 0x0000_2000_0000_0000
- Block size = 2 MB
- AF = 1
- SH = Inner Shareable
- AttrIndx = 1 (Normal NC)

****

****

5.21 — Chapter Summary

****

****

This chapter covered:

- page table hierarchy
- descriptor formats
- block vs page mappings
- AF, AP, PXN/UXN, SH, AttrIndx
- physical address encoding
- table descriptor encoding
- walk behavior
- fault types
- full examples